OpenZeppelin | security

Stellar Contracts RC v0.8.0 Audit



October 20, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 8

Security Model and Trust Assumptions 10

Medium Severity 12

M-01 Capped Vault Composition Never Checks The Cap 12

M-02 Minted Tokens Become Permanently Non-Transferable And Non-Burnable 13

Low Severity 14

L-01 Unpinned And Ungated Dependency Declarations 14

L-02 Misleading Documentation 14

L-03 Votes Extension Trait Is Declared Without A Contract Type Bound 17

L-04 Eager Fee Collection Draws On The Forwarder's Own Balance For Non-Conserving Fee Tokens 17

L-05 Redeeming Shares Burns Them For Nothing After A Clawback-Enabled Asset Is Removed From The Vault 18

L-06 Recovered Investor Balance Can Be Misdirected To A Reassigned Target 19

Notes & Additional Information 20

N-01 Swap-And-Pop Removal Splits The Lifetimes Of A Moved Fee Token's Allowlist Entries 20

Client Reported 21

CR-01 Delegated Signers Pass Without Keys When the Delegate Is the Direct Caller 21

CR-02 get_approved Function Does Not Fail As Documented 22

CR-03 Non-ZK Proofs Leak Private Balances and Enable Spending-Key Recovery 22

Conclusion 24

Appendix 25

Issue Classification 25

Summary

Type	Library
Timeline	From 2026-09-14 To 2026-09-29
Languages	Rust

Total Issues	12 (9 resolved, 1 partially resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	2 (2 resolved)
Low Severity Issues	6 (4 resolved, 1 partially resolved)
Notes & Additional Information	1 (1 resolved)
Client Reported Issues	3 (3 resolved)

Scope

OpenZeppelin conducted a diff audit of the OpenZeppelin/stellar-contracts repository covering the changes between base snapshot fbfde38 and target snapshot a5bd8cb.

In scope were the following modified or added files:

```
packages
    ├── access/src/access_control
    │   └── storage.rs
    ├── accounts/src
    │   ├── policies
    │   │   └── spending_limit.rs
    │   ├── smart_account
    │   │   ├── mod.rs
    │   │   └── storage.rs
    │   └── verifiers
    │       └── webauthn.rs
    ├── contract-utils/src
    │   ├── math
    │   │   └── wad.rs
    │   └── upgradeable
    │       └── storage.rs
    ├── fee-abstraction/src
    │   └── storage.rs
    ├── governance/src/governor
    │   ├── mod.rs
    │   └── storage.rs
    └── tokens/src
        ├── confidential/verifier
        │   ├── mod.rs
        │   └── storage.rs
        ├── fungible
        │   └── storage.rs
        ├── non_fungible
        │   ├── extensions
        │   │   ├── consecutive
        │   │   │   └── storage.rs
        │   │   └── enumerable
        │   │       └── storage.rs
        │   └── storage.rs
        ├── rwa
        │   ├── compliance/modules/max_balance
        │   │   └── storage.rs
        │   ├── extensions/doc_manager
        │   │   └── storage.rs
        │   ├── identity_verification
        │   │   ├── claim_topics_and_issuers
        │   │   │   ├── mod.rs
```

```
        │   │   │   └── storage.rs
        │   │   ├── mod.rs
        │   │   └── storage.rs
        │   ├── mod.rs
        │   └── storage.rs
        └── vault
            └── storage.rs
```

OpenZeppelin additionally conducted a diff audit of the same repository covering the changes between base snapshot 9c5e279 and target snapshot 2128f11.

In scope were the following modified or added files:

```
packages
    ├── accounts/src
    │   ├── smart_account
    │   │   ├── mod.rs
    │   │   └── storage.rs
    │   └── lib.rs
    ├── contract-utils/src/math
    │   ├── i256_fixed_point.rs
    │   ├── mod.rs
    │   └── wad.rs
    ├── fee-abstraction/src
    │   ├── lib.rs
    │   └── storage.rs
    └── tokens/src
        ├── confidential
        │   ├── auditor
        │   │   └── mod.rs
        │   ├── circuits
        │   │   ├── clawback
        │   │   │   ├── src
        │   │   │   │   └── main.nr
        │   │   │   └── Nargo.toml
        │   │   ├── gadgets
        │   │   │   ├── assert_on_curve/src
        │   │   │   │   └── main.nr
        │   │   │   └── sponge_squeeze_2/src
        │   │   │       └── main.nr
        │   │   ├── lib/src
        │   │   │   └── lib.nr
        │   │   ├── register/src
        │   │   │   └── main.nr
        │   │   ├── scripts
        │   │   │   └── extract_vks.sh
        │   │   ├── set_spender/src
        │   │   │   └── main.nr
        │   │   ├── spender_transfer/src
        │   │   │   └── main.nr
        │   │   ├── transfer/src
        │   │   │   └── main.nr
        │   │   ├── withdraw/src
        │   │   │   └── main.nr
```

```
│   │   ├── constraints.baseline
│   │   └── Nargo.toml
│   ├── compliance
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── verifier
│   │   ├── mod.rs
│   │   └── storage.rs
│   ├── mod.rs
│   └── storage.rs
├── fungible
│   ├── extensions
│   │   ├── allowlist
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── blocklist
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── burnable
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── capped
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── combinations
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── total_supply
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   └── mod.rs
│   ├── mod.rs
│   ├── overrides.rs
│   └── storage.rs
├── non_fungible
│   ├── extensions
│   │   ├── burnable
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── combinations
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── consecutive
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── enumerable
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── royalties
│   │   │   ├── mod.rs
│   │   │   └── storage.rs
│   │   ├── votes
│   │   │   └── storage.rs
│   │   └── mod.rs
│   ├── mod.rs
│   └── overrides.rs
```

```
        ├── rwa
        │   ├── identity_verification/identity_registry_storage
        │   │   ├── mod.rs
        │   │   └── storage.rs
        │   ├── mod.rs
        │   └── storage.rs
        └── vault
            ├── mod.rs
            └── storage.rs
```

System Overview

OpenZeppelin Stellar Soroban Contracts is a library of reusable contract modules for the Stellar network, distributed as a set of Rust crates rather than as a deployed protocol. It provides implementations of common token standards, smart account primitives, governance components, access control, and supporting utilities, which downstream developers compose into their own Soroban contracts. Because the library is consumed as source rather than invoked as a deployment, most of its security properties are realized only in combination with the integrating contract, and several authorization decisions are deliberately left to that contract.

The two reviewed ranges differ in character. The first is narrow in functional terms, containing two behavioral corrections, one prior-phase component re-examined without having changed, documentation additions, and comment reflowing from a formatting pass, together with two changes carried as proposed work not yet merged into its target branch. The second is substantially larger and introduces a uniform composition model across the token modules, in which each token implementation selects exactly one `ContractType` that determines all overridable behavior. The `ContractOverrides` trait routes every method of the base token trait through that selection, which either runs the default logic or substitutes an extension-specific variant, and `Compose` resolves the curated combinations. This routing is the mechanism by which transfer restrictions reach the code paths that move value, which makes the completeness of that routing a security property rather than an organizational one.

The subsystems in scope across both ranges are the following.

- **Fungible and Non-Fungible Tokens:** the SEP-41 fungible base and its non-fungible counterpart, with the allowlist, blocklist, burnable, capped, enumerable, consecutive, royalties, and votes extensions, all now routed through the composition model described above.
- **Total Supply Extension:** a new extension holding the supply counter that previously sat in the fungible base. Tracking supply is not required by SEP-41, and a shared counter serializes otherwise independent transactions under parallel execution, so it becomes opt-in.

- **Vault:** ERC-4626 share accounting, gaining a configurable virtual decimals offset, an effective asset denominator, and rejection of a positive deposit that would mint zero shares.
- **Real World Asset Token:** the ERC-3643 (T-REX) operator-gated entry points gain atomic batch siblings, and recovery now carries a full address freeze to the destination even when the source wallet holds no tokens.
- **Identity Verification:** the claim issuer, claim topic registry, identity, and registry storage contracts behind holder eligibility. Registry storage is modified, and the tree gains documentation asserting behavior the implementation is expected to match.
- **Confidential Token:** the token, verifier, and auditor registry contracts, holding balances as Pedersen commitments on Grumpkin and requiring an UltraHonk proof on every operation that consumes private state. The compliance extension gains opt-in seizure, and spender revocation no longer carries a proof.
- **Clawback Circuit:** a new Noir circuit supporting the stated seizure. It proves that a compliance seizure of a public amount is bounded by what the target actually holds, without revealing either balance, and that the prover holds the auditor key the target's account is bound to.
- **Smart Account and Policies:** context rules binding signers and policies to operation scopes. Signers now commit to a digest binding the signature payload, the account address, and the rule selection. The spending limit policy's retained history cap is reduced to the largest size that fits a single ledger entry.
- **Governor:** proposal creation, voting, counting, and execution across a lifecycle delimited by ledger-based timing parameters. A zero voting period sets the vote end equal to the vote snapshot, leaving a window that no ledger falls inside, so every proposal moves from pending straight to defeated without ever accepting a vote. The setter is typically reachable only through a passing proposal or the constructor, so a governor left in that state would be beyond recovery, and zero is now rejected.
- **Fee Abstraction, Upgradeability and Fixed-Point Math:** fee settlement in a fungible token through a forwarder, contract upgrade support, and multiply-then-divide primitives with explicit rounding directions over 128-bit and 256-bit signed integers.

Security Model and Trust Assumptions

The assumptions below are recorded in addition to previous reports, which remain in force and are not restated, while others follow from the changes in this scope's audit.

- **Shared storage keyspace:** A `#[contracttype]` storage key encodes to its variant name and fields, never to the name of the enum that declares it, so library modules share one keyspace with each other and with the integrator's own keys. Deployments are assumed not to compose modules whose keys alias, namely the fee forwarder and smart account `Count` counters in instance storage and the fungible and non-fungible `Balance(Address)` entries in persistent storage, and not to declare a variant matching a library key such as `TotalSupply` in the same tier. The fix is expected from the SDK rather than the library: stellar/rs-soroban-sdk#1970 qualifies user-defined type names by module path, and stellar/rs-soroban-sdk#2070 records the related absence of enum-name checking. Neither is merged, and `#1970` revises the contract specification only, leaving ledger keys unchanged. Qualifying storage keys by module at a later date would orphan the state of every deployed contract unless a migration ships with it. stellar/rs-soroban-sdk#2088 is an open discussion that touches the issue in question.
- **Relocated total supply:** The supply counter moved from instance to persistent storage when supply tracking became an opt-in extension in OpenZeppelin/stellar-contracts#795. A token upgraded across that boundary reads the new key, finds nothing, and reports a supply of zero while the old entry remains in place. No migration helper ships with the release. The library is encouraged to add guidance in the release notes, so integrators upgrading an existing token are assumed to move the value themselves.

Update: *The library team delivered a migration helper for the total supply storage location change in pull request #933 at commit bc81afe.*

- **Fee forwarder authority:** `collect_fee_and_invoke` performs the forwarded call itself, so that call executes with the forwarder's own authority over any fees it holds and any allowance granted to it. The `Lazy` strategy, and any configuration in which the forwarder is its own fee recipient, is intended only for permissioned deployments whose invoker is access-controlled and screens each forwarded call for that authority.
- **Smart account rule set:** `remove_context_rule` applies no lower bound on how many rules remain, and `update_context_rule_valid_until` rejects only an expiry

already in the past, so an account can be left holding no valid rule. Every administrative entry point resolves a rule, so such an account can no longer authorize anything, including adding a rule back. This is accepted as a feature the library does not undertake to prevent, there being no burn address on Stellar to which ownership could otherwise be transferred.

- **Vault asset custody:** `FungibleVault` pins its contract type to `Vault`, and every default body forwards to that concrete type, so an override replaces the exported entry point while the vault's internal accounting continues to use the library value. A vault holding its assets in an external strategy contract has to override `total_assets`, which the conversion and preview paths consume directly. Vaults of that shape are assumed to fall outside what the library currently supports.

- **Delegated spenders:** A freeze constrains an account's own funds and not its capacity to act as a delegate, so a frozen account holding an allowance can still move an unfrozen owner's tokens. The confidential token applies its external policy check to the spender but not its freeze or Stellar Asset Contract checks.

Medium Severity

M-01 Capped Vault Composition Never Checks The Cap

`Capped` is modelled as purely additive, so listing it in a composition does not change the contract type the list resolves to. A list naming `Vault`, `Capped` and `TotalSupply` therefore compiles and resolves to `Vault`, as the library's own composition test asserts. `FungibleCapped` requires only `FungibleTotalSupply`, with no constraint on the contract type, and its `cap` default reads the stored cap directly. As a result, a vault share token can implement the trait and publish a `cap()` value without any compile or runtime error.

That cap binds nothing. `check_cap` is called only inside `Capped::mint`, which no vault path uses. Shares are created by `Vault::deposit` and `Vault::mint`. Both go through `deposit_internal`, where the supply is increased without reference to the cap. Both entry points authorize only the caller-supplied `operator`, so any account can deposit and mint shares past the advertised ceiling while `cap()` keeps reporting it. The capped module notes that a contract type with its own mint should call `check_cap` first. However, the vault's mint belongs to the library, not the integrator, and it does not make that call.

Consider rejecting a composition that names both `Vault` and `Capped` at compile time through a dedicated error row alongside the existing `SupplyRequirementMet` and `SupplyListedOnce` rows, consistent with the module's stated design that invalid lists do not compile. Alternatively, call `check_cap` from `deposit_internal` whenever a cap has been set.

Update: *Resolved in pull request #913 at commit 486bfeb. `Capped` is now a contract type of its own, and a composition of `Vault`, `Capped`, and `TotalSupply` resolves to a new `CappedVault` type that checks the cap on `deposit` and `mint` and reflects it in `max_deposit` and `max_mint`. In addition, `FungibleCapped` can now only be implemented by contract types that enforce the cap, so a contract can no longer report a cap it does not enforce. The same approach was extended to the allowlist, blocklist, and RWA contract types. Calling a concrete type's mint directly (e.g., `Vault::deposit` on a capped vault) still bypasses the cap, but minting through the contract type is the documented path, and no library documentation or example does otherwise.*

M-02 Minted Tokens Become Permanently Non-Transferable And Non-Burnable

Every overridable non-fungible operation dispatches through the token's contract type, as `transfer` forwarding to `Self::ContractType` shows. Minting is the exception. The overridable surface is declared across `ContractOverrides`, `BurnableOverrides` and `RoyaltySupport`, none of which declares a mint, so a mint site names a concrete type directly and nothing verifies that choice against the type the composition resolves to.

Each contract type carries bookkeeping its transfer and burn paths assume, and minting through a type below the resolved one omits it. `Base::sequential_mint` leaves a later transfer reverting for `Enumerable`, `Consecutive` and `NonFungibleVotes` alike, raising `TokenNotFoundInOwnerList` or `InsufficientVotingUnits`. The mint performing both halves for `Compose<(Enumerable, NonFungibleVotes)>` is defined on `EnumerableVotes`, imported privately rather than re-exported and unnameable from an integrating contract.

Consider adding a mint trait alongside the existing override traits, implementing it for every contract type, and reducing the visibility of the inherent mint functions so that a mismatched mint no longer compiles.

Update: *Resolved in pull request #911 at commit 3b4b8f1. The team stated:*

> *Minting now dispatches through the contract type like every other overridable operation. Each non-fungible contract type exposes its mint primitives under the same names, backed by new override traits implemented for every contract type.*
>
> *On the suggestion to reduce the visibility of the inherent mint functions: we kept them public deliberately. They are the primitives the contract type resolves to (the override trait impls delegate to them), they are used directly by library code and tests, and narrowing them would be a large breaking change across the library. The contract-type path is documented as the way to mint, and it is the only one used in the examples.*

Low Severity

L-01 Unpinned And Ungated Dependency Declarations

The zero-knowledge backend for the confidential token is introduced as an ordinary dependency of the `stellar-tokens` crate rather than behind a cargo feature, which couples three separate concerns to every consumer of the package regardless of which token they use.

- The `ultrahonk-soroban-verifier` entry is a git dependency pinned to a fork, while the package is marked publishable. Cargo refuses to publish a crate carrying a git dependency, so a release of `stellar-tokens` cannot be cut while the entry is present.
- The `alloc` feature added to `soroban-sdk` exists to satisfy that backend. Cargo features are additive, so a contract depending on `stellar-tokens` for a plain fungible token or a non-fungible token registers a global allocator in its `wasm32v1-none` build and links a pre-release verifier that the module's own documentation describes as unaudited and not production ready.
- The `soroban-poseidon` patch points at a fork by mutable branch rather than by revision, under the same version string the workspace requests from the registry. That crate computes the domain-separated address commitment consumed as a public input to the proofs, so the code on the proof-binding path is not pinned and a force-push to the branch changes it at the next dependency resolution. The lockfile records a revision today and the patch does not propagate to published consumers, which confines this to the workspace build.

Consider gating the confidential token surface behind a cargo feature so the verifier and `alloc` are opt-in, pinning `soroban-poseidon` by revision rather than branch, and adding a publish dry run to CI so the release blocker surfaces before a version cut.

Update: *Resolved in pull request #922 at commit 06a3314.*

L-02 Misleading Documentation

Doc comments are the surface an integrator reads before writing a contract against the library, and several of them misstate or omit what the code does, or describe host behavior that no longer holds under `soroban-sdk` 28.

The following are instances of misleading documentation:

- The `AllowBlockList` header states that an account has to be allowed and must not be blocked in order to send, receive, approve or burn tokens, and the `AllowListVotes`, `BlockListVotes` and `AllowBlockListVotes` headers each state that every mint, transfer and burn goes through the list policy. The mint bodies of the three votes types forward straight to `FungibleVotes::mint`, which credits the recipient and moves voting units without consulting either list, and `AllowBlockList` declares no mint at all, leaving a mint to reach the base implementation unchanged. A token deployed on these headers can credit an address that was never allowed, or one that has been explicitly blocked.
- The doc block on the `TotalSupplyOverrides` hook tells the reader that implementing `FungibleTotalSupply` with an empty body is enough because the library ships the override for its supply-aware contract types, and it names `FungibleVotes` and the combined contract types resolved by `Compose` among them. `FungibleVotes` implements only the base and burnable hooks, and of the seven combined contract types only the three supply-tracking ones carry the implementation. The `FungibleTotalSupply` module documentation records the opposite, leaving a reader planning a votes-based token with two contradictory accounts of which contract types report a supply.
- The `# Events` block on `recover_balance` names the transfer and the recovery success event only, and presents itself as a complete account of what the function publishes. Three further events reach the ledger. Where the old account carries a partially frozen balance, the shared privileged transfer helper publishes a `tokens_unfrozen` event as it breaks the freeze to move the balance, and `freeze_partial_tokens` publishes a `tokens_frozen` event when that amount is re-applied to the new account. Where the old account was frozen at the wallet level, `set_address_frozen` publishes an `address_frozen` event against the new account. An off-chain system that reconstructs freeze state from the documented set observes none of them, so its view of the new account's restrictions goes stale.
- The `FungibleVault` trait warns that `deposit`, `mint`, `withdraw` and `redeem` have no authorization controls, while the trait pins its contract type to `Vault` and every default body forwards there, where the operator is authorized first. An integrator who follows the warning adds a second authorization request for the same address in one invocation, which Soroban rejects, so the resulting entry point always fails.
- The documentation for `Wad::exp` states that the function panics for inputs at or above approximately `135.305`. That figure is the limit of the algorithm's internal i256 representation rather than the largest input whose result fits the public `i128` type, which the final conversion puts at approximately `46.5832`. Every input between the two

values returns `None` from `Wad::checked_exp` and panics in `Wad::exp`, against a documented ceiling almost three times higher.

- The accounts module's caveats section warns that removing signers from a rule can leave its threshold policy permanently unsatisfiable, but does not record a second way an administrator can permanently lock the account. `remove_context_rule` applies no lower bound on how many rules remain, and `update_context_rule_valid_until` rejects only an expiry in the past, so the last rule can be removed outright or given a `valid_until` that later lapses. Every administrative entry point authorizes through `do_check_auth`, which resolves a context rule id, so an account left holding no valid rule can no longer authorize anything, including adding a rule back. The `get_context_rules_count` documentation meanwhile records a count of zero as an ordinary default, with nothing to mark it as terminal.
- The upgradeable module's migration guidance tells integrators that the host validates a stored structure's field count before the SDK sees the value, and names the specific error a shape mismatch produces. Under `soroban-sdk` 28, unpacking a `contracttype` structure with named fields is tolerant: a field absent from the stored map unpacks as void, resolving to `None` for an optional field, and a key present in the map but absent from the structure is discarded silently and lost if the value is later written back. The SDK notes there is no way to request the earlier strict behavior. The section's own example adds a non-optional field, which still fails, so its outcome holds while the stated reason and the named error do not.
- The documented data shapes for the context rule events list the optional expiry field unconditionally, while `soroban-sdk` 28 omits void fields from an event's data map by default unless the event sets `sparse = false`, which no event in the library does. A consumer coded against the documented shape finds the key absent rather than present and void whenever no expiry is set, which is the ordinary case.
- The `set_address_frozen` documentation states that frozen wallets cannot send or receive funds, without distinguishing a wallet's own balance from its capacity to act as a delegate, which `transfer_from` does not check, and `recover_balance` reports success while allowances granted to the old account survive it.

Consider fixing the misleading documentation to reflect the code's behavior and avoid misimplementation by users.

Update: *Resolved in pull request #932 at commit 30b0fe4.*

L-03 `Votes` Extension Trait Is Declared Without A Contract Type Bound

Both composition modules promise that implementing an extension trait the list does not back is rejected by that trait's bound (fungible, non-fungible). Eight extension traits are declared that way, binding the associated contract type to a marker, among them `NonFungibleEnumerable`, `FungibleAllowList` and `FungibleTotalSupply`.

`Votes` is the exception. It is declared bare, with no supertrait and no constraint on the associated contract type. No marker trait over the `FungibleVotes` and `NonFungibleVotes` contract types exists to bind it against.

A contract can therefore resolve to `Compose<(AllowList,)>`, implement `Votes` and compile. Transfers, mints and burns never reach `transfer_voting_units`, so no checkpoint is written. `get_votes` and `get_votes_at_checkpoint` return zero for every account at every ledger. `delegate` succeeds and emits its event while moving nothing. The contract type is fixed at compile time, so this holds for the life of the contract.

Consider adding a votes marker trait in the tokens package, implemented by the contract types that move voting units, and bounding a token-side votes trait on it. Otherwise, name `Votes` in both composition modules as an explicit exception.

Update: *Acknowledged, will resolve. The team stated:*

> *Acknowledged, will resolve later. Tracked in issue #930.*

L-04 `Eager` Fee Collection Draws On The Forwarder's Own Balance For Non-Conserving Fee Tokens

The `Eager` branch of `collect_fee` pulls `max_fee_amount` from the user into the forwarder, pays `fee_amount` to the recipient and refunds the remainder, so the amount paid out is always exactly `max_fee_amount` regardless of how much was received. Where a fee token withholds part of a transfer, as fee-on-transfer, deflationary and rebasing tokens do, the difference is taken from the forwarder's own balance, and the collection either succeeds while silently reducing that balance or panics and reverts when the balance is insufficient.

Consider moving `fee_amount` directly from the user to the recipient and then setting the allowance to zero, which leaves no residual allowance without placing the forwarder's balance at risk. Under the same tokens, `emit_fee_collected` also reports the requested

`fee_amount` rather than the amount the recipient received, and `emit_tokens_swept` reports the balance measured before the transfer, so consider deriving both from the recipient's measured balance change to keep the emitted values consistent with what was actually moved.

Update: *Partially Resolved in pull request #919 at commit 334aeee. The team stated:*

> *We went with a docs fix: `Eager` is now documented as requiring a fee token that credits the receiver exactly what it debits from the sender, and `Lazy`, which never touches the forwarder's balance, is the recommended strategy for fee-on-transfer and other non-conserving tokens. We didn't adopt the direct transfer + `approve(0)` flow because the extra `approve` needs a second user-signed node in the auth tree, which would change the signing payload for every `Eager` integration. Without it, the leftover allowance would make `Eager` behave like `Lazy`. The docs now also note that `fee_collected` reports the requested `fee_amount`. We'd keep `tokens_swept` as is, since it reports the amount taken from the forwarder, which is what that event is meant to show.*

L-05 Redeeming Shares Burns Them For Nothing After A Clawback-Enabled Asset Is Removed From The Vault

`redeem` computes the assets due with `Rounding::Floor` and passes the result to `withdraw_internal`, which burns the shares and transfers whatever amount it was given without checking that the amount is positive. The quotient is zero whenever `shares` multiplied by `total_assets + 1` falls below `total_supply + 10^offset`, which is the case for smaller holders whenever the vault holds few assets, and for every holder once the asset balance reaches zero. Since `total_assets` reads the vault's live balance, an underlying asset supporting clawback or forced transfers can reach that state without any vault function being called. The shares are destroyed and the supply reduced while the redeemer receives nothing, so the remaining shares rise in value and the loss passes to the holders who did not redeem.

Consider rejecting a redemption of a positive number of shares that would return zero assets, mirroring the check already performed on deposit.

Update: *Resolved in pull request #921 at commit c1449c4.*

L-06 Recovered Investor Balance Can Be Misdirected To A Reassigned Target

Account recovery spans two contracts and two transactions. The registry first moves the identity and records a pointer from the old wallet to the new one. The token contract then moves the balance, gated only on that pointer, as `RWA::recover_balance` requires the recorded target to match its destination. Nothing reserves the destination in the interval.

If the registry operator removes and reassigns the destination within that interval, no guard prevents it. The balance check sweeps the account being removed, which is empty because the tokens are still on the old wallet. The recovered-account check reads the pointer keyed on the account being registered, while it was written on the old wallet. The guard against overwriting a live registration lapses once the entry is removed. The second step then transfers the balance into a wallet that now belongs to a different investor, and reversing it takes a forced transfer that the maximum balance module can reject, since it recorded neither side of the misdirected move. While this requires an operator error, the registry states that a wallet used as a recovery target cannot be reused, which the code does not enforce.

Consider storing the identity alongside the recovery pointer and requiring the destination of `recover_balance` to still resolve to it, and aligning the registry documentation with the reuse rule the code enforces.

Update: *Resolved in pull request #924 at commit bcd9a70. The team stated:*

> *Resolved with a different approach from the suggested one.*
>
> *Instead of detecting the reassignment in `recover_balance`, the registry now prevents it. `recover_identity(A, B)` also records a reverse link, `RecoveredFrom(B) -> A`.*

Notes & Additional Information

N-01 Swap-And-Pop Removal Splits The Lifetimes Of A Moved Fee Token's Allowlist Entries

An allowed fee token is held as two persistent entries, `TokenIndex(token)` and `Token(index)`, and a successful lookup in `is_allowed_fee_token` extends both, so under ordinary use the two share a lifetime. Removing a token other than the last moves the last token into the vacated slot by overwriting `Token(remove_index)` and `TokenIndex(last_token)`. An overwrite keeps the existing entry's expiry, so the moved token's slot entry inherits the removed token's remaining lifetime while its `TokenIndex` entry keeps its own, and no extension is applied on this path.

When the removed token had been idle longer than the moved one, the moved token's slot entry archives first. The contract never observes the archived entry: a simulated collection restores it automatically at the cost of a restore fee and a larger footprint, while one submitted with a stale footprint is rejected before execution with `INVOKE_HOST_FUNCTION_ENTRY_ARCHIVED`. The next successful lookup extends both entries and brings them back in step.

Consider extending `Token(remove_index)` and `TokenIndex(last_token)` when the removal path rewrites them, so that the moved token's entries leave the removal with a common lifetime.

Update: *Resolved in pull request #917 at commit 652a247.*

Client Reported

CR-01 Delegated Signers Pass Without Keys When the Delegate Is the Direct Caller

When `require_auth` targets a contract address, the Soroban host runs that contract's `__check_auth`, unless the contract is the direct caller of the contract requiring authorization. In that case the host approves it without checking any signature.

`authenticate` checks a `Signer::Delegated` signer by calling `require_auth_for_args` on the delegate. If the delegate is the direct caller of the account's `__check_auth`, it passes with empty signatures. If it alone satisfies the selected rule (sole signer, or enough for a threshold policy), anyone can use the account for whatever that rule covers. For a default rule, this means draining funds, adding signers, or taking over the account.

The attacker cannot create this setup. Signers are set at deployment or through functions such as `add_context_rule` and `add_signer`, which require the account's own authorization. This includes an account adding itself as a `Delegated` signer. The risk is an owner choosing such a setup in good faith, for example two accounts as each other's backup.

Below, "`A` delegates to `B`" means that smart account `A` lists `Delegated(B)` as a signer. The affected setups are:

- `A` delegates to `B`, and `B` delegates to `A`: `A` is the direct caller of `B`'s check, so `B`'s request for `A` passes.
- `A` delegates to `B`, `B` to `C`, and `C` back to `B`: `B` is the direct caller of `C`'s check, so `C`'s request for `B` passes.
- `A` delegates to itself: when one of `A`'s own functions, such as `execute`, requires `A`'s authorization, `A` is the direct caller of its own check.

The delegate does not have to be an account: a contract listed as a `Delegated` signer of `A` is the direct caller whenever it calls `A.require_auth` itself. A loop through three accounts, where `A` delegates to `B`, `B` to `C`, and `C` back to `A`, is not affected: `B`, not `A`, is the direct caller of `C`'s check, and the host does not approve `A` again while its own check runs.

`get_validated_context_by_id` does not stop these setups: its signer check only confirms that the rule's signers are listed in the payload, not that they signed.

Partners that use CAP-71 delegation hit the same flaw in a form the smart account cannot detect. Since protocol 28, a custom account `D` can call `delegate_auth` inside its `__check_auth` to let `A` authenticate for it. The host then runs `A.__check_auth` directly from `D.__check_auth`, with `D`'s signature payload and contexts. If `A` lists `Delegated(D)`:

- `D` is the direct caller of `A.__check_auth`, but nothing `A` receives says so. Its contexts describe what `D` authorizes, not who called `A`.
- `A` calls `D.require_auth_for_args`, and the host approves it as the direct caller.
- `D` is authorized without any key. The victim is `D`, not `A`.

Consider rejecting a `Delegated` signer that is the direct caller of `__check_auth`, for example by checking it against the authorization contexts, where the direct caller usually appears as the contract of the root context. Consider also documenting that a `Delegated` signer must not be an account that delegates its own authentication to the smart account through CAP-71 `delegate_auth`, nor one whose `__check_auth` calls `require_auth` on the smart account without the same check, since the smart account cannot detect either.

CR-02 `get_approved` Function Does Not Fail As Documented

The `get_approved` function's documentation mentions that it should fail if the token ID passed as parameter does not exist. However, `ContractOverrides::get_approved` calls `Base::get_approved`, and neither of them fails when the token ID does not exist.

Consider adding a token existence check to the `ContractOverrides::get_approved` surface so that each extension enforces it against its own ownership storage, as `token_uri` already does.

Update: *Resolved in pull request #935 and pull request #944 at commit ca4f88d.*

CR-03 Non-ZK Proofs Leak Private Balances and Enable Spending-Key Recovery

The Pedersen generators `G` and `H` are compile-time constants matching Barretenberg's precomputed fixed-base generators. The `commit` and `scalar_mul` functions pass them directly to `multi_scalar_mul`, which Barretenberg v0.87.0 lowers to fixed-base lookup tables indexed by nine-bit windows of private scalars. Since the proving configuration omits

`--zk`, `lookup_read_counts` and `lookup_read_tags` are committed without masking. Their commitment difference is supported on rows accessed multiple times, making the resulting scalar-dependent collisions recoverable through a meet-in-the-middle search.

A passive observer can exploit this leakage using only public proofs. Shared high-order digits of the original and updated balances create detectable lookup collisions, revealing a median of nine balance bits per proof in testing. More critically, the spending key `sk` appears across an account's proofs while the other contributing scalars vary with fresh randomness and evolving blindings, allowing collisions involving `sk` to accumulate. In one measured account history, key-recovery work decreased from approximately 2^46 at 400 proofs to 2^16 at 700 proofs. The recovered key was verified against `Y = sk · H` and used to decrypt all 700 transfer amounts and balances. Historical proofs remain exposed after remediation, although recovering `sk` alone does not bypass Stellar's `require_auth` authorization.

Consider passing `G` and `H` as constrained witnesses to force ROM-backed variable-base multiplication, eliminating the demonstrated fixed-base lookup leakage. A CI check should also reject constant-base MSMs and blackboxes that introduce witness-dependent lookups. This closes the identified leakage channel but does not guarantee zero knowledge property. Full witness confidentiality requires a ZK proving configuration and compatible verifier.

Update: *Resolved in pull request #945 at commit 062e5ef.*

Conclusion

The OpenZeppelin Stellar Contracts library provides reusable token, account, and utility modules that developers compose into their own Soroban contracts. OpenZeppelin audited two ranges of changes: the 0.8.1 line on `main`, which carries two narrow behavioral fixes, and the v0.9.0 branch, which introduces a uniform contract-type composition model across the token modules, confidential token seizure with its supporting circuit, real-world asset batch operations, and optional supply tracking.

Two themes run through the findings. The composition model concentrates all overridable behavior in a single contract type, which is a clear improvement over per-method rewiring, but its type-level guarantees do not yet reach minting or the vault and real-world asset combinations, so a token can publish a control that the resolved type never enforces.

Several issues were dismissed as intentional behavior, and were documented in the security model of this report. We advise the team to document these decisions in detail, as well as issues that might arise from upstream dependencies to ease integration and avoid confusion.

We thank the OpenZeppelin Stellar Contracts team for their responsiveness throughout the engagement, and for the detailed answers they provided on the design decisions behind the behaviors described above.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.